NATIONAL UNIVERSITY

NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

Laboratory Activity 3
RESTful API Design Standards

Name	Marzan, Joshua M.
Section	INF 233
Date	15 October 2026

Objectives

- Develop RESTful APIs that follow resource-based design principles.
- Implement CRUD operations for Bulldogs Exchange resources using Express.js and MongoDB.
- Apply appropriate HTTP methods and status codes in API development.
- Implement pagination, filtering, and sorting for product resources.
- Test RESTful API endpoints using Postman/Insomnia REST.

Materials Needed:

- Visual Studio Code
- Node.js (LTS Version)
- MongoDB Community Server or MongoDB Atlas
- Postman/Insomnia REST

Prerequisite(s):

- Completed Week 3 Laboratory Activity (Advanced Database Design Using MongoDB)
- Existing Bulldogs Exchange backend project with completed Mongoose schemas
- Basic understanding of RESTful API architecture and HTTP methods

Submission Requirements:

- Filename: Surname_Activity2_CTADWEBL
- Save as PDF

Activity Instructions

Part 1: Preparing the API Structure
Open the Bulldogs Exchange backend project completed during Week 3. **Verify that (***Provide the Screenshot of your Backend Working*`)`**:** • The application runs successfully. • The MongoDB connection is established. • All Mongoose schemas load without errors.

The Bulldogs Exchange backend (**marzan-server**) from Week 3 was started with `npm run dev`. The console shows that the MongoDB Atlas connection is established and that every Mongoose schema loaded without errors: right after connecting, `db.js` prints the registered models with `mongoose.modelNames()`.

```
> npm run dev
MongoDB connected
Models loaded: Product, Category, Supplier, User, Cart, Order, Review
Server running on http://localhost:5000
```

- **The application runs successfully:** `GET http://localhost:5000/` returns `{ "success": true, "message": "BulldogEx Shop API is running", "data": { "version": "v1" } }`.
- **The MongoDB connection is established:** the console prints `MongoDB connected` (database `bulldogex-shop` on MongoDB Atlas).
- **All Mongoose schemas load without errors:** Product, Category, Supplier, User, Cart, Order, and Review are listed. Supplier is new in this activity because Part 5 tests suppliers and Part 7 filters products by supplier.

Screenshot below: VS Code with the terminal output of `npm run dev`.

Part 2: Applying RESTful API Standards

Ensure all endpoints follow RESTful naming conventions (Provide screenshots each of your routes and your server/index.js file, add more pages if necessary**):**

Examples:
✓ /api/product
✓ /api/category
✓ /api/cart
✓ /api/order
✓ /api/review
✓ /api/user

All endpoints now follow RESTful naming conventions:

- URLs name **resources with plural nouns**: products, categories, suppliers, users, sessions, carts, orders, reviews.
- The **HTTP method** expresses the action: GET reads, POST creates, PUT updates, DELETE deletes. No URL contains a verb.
- A single resource is addressed by its identifier: `/api/v1/categories/:id`. Products use their unique URL name (slug), e.g. `/api/v1/products/hoodie-jacket`, which is the same identifier the

storefront pages use.

- Every route is **versioned** under `/api/v1`, and query parameters are used for options such as `?page=1&limit=10`.

Plural nouns (`/api/v1/products`) are used instead of the singular examples (`/api/product`) because each URL names a collection of resources. This is the common REST convention and matches the query examples in Part 4 (`GET /api/products?page=1&limit=10`).

Changes made in this activity

Before (Activities 1–2)	After (Activity 3)	Reason
/api/products, /api/categories, ...	/api/v1/products, /api/v1/categories, ...	Every route is versioned under /api/v1 (Part 7).
POST /api/users/signup	POST /api/v1/users	"signup" is a verb. Creating an account is a POST to the users collection.
POST /api/users/signin	POST /api/v1/sessions	"signin" is a verb. Signing in creates a session (a JWT), so it is a POST to sessions.
GET /api/users/profile	GET /api/v1/users/me	The signed-in user's own user resource.
(none)	/api/v1/suppliers	New Supplier resource with full CRUD.
Categories: GET and POST only	GET, POST, PUT, DELETE /api/v1/categories	Full CRUD so categories can be maintained.

Final API endpoints

Method	Endpoint	Action	Auth
GET	/api/v1/products	List products (pagination, filters, sort, search)	–
GET	/api/v1/products/:name	Get one product by its URL name	–
POST	/api/v1/products	Create a product	Bearer token
PUT	/api/v1/products/:name	Update a product	Bearer token
DELETE	/api/v1/products/:name	Delete a product	Bearer token
GET	/api/v1/categories	List categories	–
GET	/api/v1/categories/:id	Get one category	–
POST	/api/v1/categories	Create a category	Bearer token
PUT	/api/v1/categories/:id	Update a category	Bearer token
DELETE	/api/v1/categories/:id	Delete a category (only if no products use it)	Bearer token
GET	/api/v1/suppliers	List suppliers	–
GET	/api/v1/suppliers/:id	Get one supplier	–
POST	/api/v1/suppliers	Create a supplier	Bearer token
PUT	/api/v1/suppliers/:id	Update a supplier	Bearer token

DELETE	/api/v1/suppliers/:id	Delete a supplier (only if no products use it)	Bearer token
POST	/api/v1/users	Sign up (create a user)	–
GET	/api/v1/users/me	Get the signed-in user's profile	Bearer token
POST	/api/v1/sessions	Sign in (create a session, returns a JWT)	–
GET	/api/v1/carts (?user=<id>)	List carts	–
GET	/api/v1/carts/:id	Get one cart	–
POST	/api/v1/carts	Create a cart	–
GET	/api/v1/orders (?user=<id>, ?status=)	List orders	–
GET	/api/v1/orders/:id	Get one order	–
POST	/api/v1/orders	Place an order	–
GET	/api/v1/reviews (?product=<id>)	List reviews	–
GET	/api/v1/reviews/:id	Get one review	–
POST	/api/v1/reviews	Write a review	–

[Paste screenshot here: index.js (all routers mounted under /api/v1)]

[Paste screenshot here: components/routes/productRoutes.js]

[Paste screenshot here: components/routes/categoryRoutes.js]

[Paste screenshot here: components/routes/supplierRoutes.js]

[Paste screenshot here: components/routes/userRoutes.js and sessionRoutes.js]

[Paste screenshot here: components/routes/cartRoutes.js]

[Paste screenshot here: components/routes/orderRoutes.js]

[Paste screenshot here: components/routes/reviewRoutes.js]

Part 3: Creating HTTP Status Code Constants

Do the following (*Provide Screenshot of your Source Code and Output, add more pages if necessary***):**

- Create a constants.js file inside the config folder
- Inside your constants folder create a HttpStatus constant
- Provide each HTTP Codes with the right naming convention
- Refactor your existing code and use your newly created HTTP Code constants.

Example Code:

```
const HttpStatus = {
  OK: 200,
  CREATED: 201,
  BAD_REQUEST: 400,
  UNAUTHORIZED: 401,
  FORBIDDEN: 403,
```

```
  NOT_FOUND: 404,
  INTERNAL_SERVER_ERROR: 500,
};

export { HttpStatus };
```

A `constants.js` file was created inside `components/config` with a `HttpStatus` object. Every code has an UPPER_SNAKE_CASE name that matches its official HTTP meaning. The whole project uses CommonJS (`require` / `module.exports`), so the constant is exported with `module.exports` instead of the ES-module `export` in the example; both produce the same object.

components/config/constants.js

```
const HttpStatus = {
  OK: 200,
  CREATED: 201,
  BAD_REQUEST: 400,
  UNAUTHORIZED: 401,
  FORBIDDEN: 403,
  NOT_FOUND: 404,
  CONFLICT: 409,
  INTERNAL_SERVER_ERROR: 500,
};

module.exports = { HttpStatus };
```

Constant	Code	Returned when
HttpStatus.OK	200	A GET, PUT, or DELETE request succeeded.
HttpStatus.CREATED	201	A POST request created a new document (user, category, supplier, product, cart, order, review).
HttpStatus.BAD_REQUEST	400	Validation errors, an invalid ObjectId, invalid query values (page, limit, sort, minPrice, maxPrice), an invalid JSON body, missing sign-in fields, or a referenced category, supplier, user, or product that does not exist.
HttpStatus.UNAUTHORIZED	401	No token, an invalid or expired token, or a wrong email or password.
HttpStatus.FORBIDDEN	403	Reserved for a signed-in user who is not allowed to do an action (included for completeness, not used yet).
HttpStatus.NOT_FOUND	404	The requested product, category, supplier, cart, order, or review does not exist, or the route does not exist.
HttpStatus.CONFLICT	409	A duplicate unique value (email, category, supplier, or product name), a second cart for the same user, a second review of the same product, or deleting a category or supplier that products still use.
HttpStatus.INTERNAL_SERVER_ERROR	500	An unexpected error on the server.

Refactoring the existing code

Every controller, the auth middleware, and `index.js` now use the constants instead of numbers. A small helper, `components/utils/apiResponse.js`, sends every response in the same JSON format and maps Mongoose errors to the right constant: ValidationError and CastError → BAD_REQUEST, duplicate key (error code 11000) → CONFLICT, anything else → INTERNAL_SERVER_ERROR.

```
// Before (Activity 2)
res.status(400).json({ message: "Category not found" });
res.status(201).json(product);

// After (Activity 3)
const { HttpStatus } = require("../config/constants");
return sendError(res, HttpStatus.BAD_REQUEST, "Category not found");
return sendSuccess(res, HttpStatus.CREATED, "Product created successfully.", product);
```

[Paste screenshot here: components/config/constants.js]

[Paste screenshot here: components/utils/apiResponse.js]

[Paste screenshot here: a refactored controller, e.g. categoryController.js using HttpStatus]

[Paste screenshot here: Output: Postman responses showing 201 Created, 400 Bad Request, 404 Not Found, and 409 Conflict]

Part 4: Implementing Advanced Query Features

Task 1: Implement pagination for the Product API (*Provide Source code and Output*).

Example:

GET /api/products?page=1&limit=10

`GET /api/v1/products` accepts **page** (default 1) and **limit** (default 10, maximum 100). The controller checks that both are whole numbers in range, skips `(page – 1) × limit` products, returns at most `limit` products, and adds `total`, `page`, `limit`, and `totalPages` to the response so the client knows how many pages exist. Results are always ordered by a stable sort (the requested field, then `_id`), so a product never appears on two pages.

```
const pageNumber = Number(page);    // default "1"
const limitNumber = Number(limit);  // default "10", maximum 100

const [products, total] = await Promise.all([
  Product.find(filter).sort(sortBy)
    .skip((pageNumber - 1) * limitNumber)
    .limit(limitNumber)
    .populate(productPopulate),
  Product.countDocuments(filter),
]);
// response: { success, message, count, total, page, limit, totalPages: Math.ceil(total /
limitNumber), data }
```

Request: `GET /api/v1/products?page=1&limit=5` (Postman 5.1) returns 5 products with `"page": 1, "limit": 5, "totalPages": 3`; `?page=3&limit=5` (5.2) returns the remaining products.

Box 1 below: getProducts pagination code in productController.js. **Box 2 below:** Postman output of GET /api/v1/products?page=1&limit=5.

Task 2: Implement filtering for the product API (*Provide Source code and Output*).

Example:

GET /api/products?category=Uniform

Products can be filtered by **category** and **supplier** (by name, case-insensitive, or by _id) and by a **price**

range (`minPrice`, `maxPrice`). Because products store references, a name is first converted to its ObjectId with `findIdByName()`. A name that does not exist returns an empty list with 200 OK.

```
if (category) filter.category = (await findIdByName(Category, category)) ?? NO_MATCH;
if (supplier) filter.supplier = (await findIdByName(Supplier, supplier)) ?? NO_MATCH;
// minPrice -> filter.price.$gte, maxPrice -> filter.price.$lte

const findIdByName = async (Model, value) => {
  const query = mongoose.isObjectIdOrHexString(value) ? { _id: value } : { name: value };
  const doc = await Model.findOne(query).collation({ locale: "en", strength: 2 }).select("_id");
  return doc ? doc._id : null;
};
```

Request: `GET /api/v1/products?category=Uniforms` (Postman 5.3). The category in this catalog is named **Uniforms**, so the request uses that name instead of "Uniform" from the example.

Box 1 below: filtering code in productController.js. **Box 2 below:** Postman output of GET /api/v1/products?category=Uniforms.

Task 3: Implement sorting for the product API (*Provide Source code and Output*).

Example:

GET /api/products?sort=price

sort accepts `price`, `title`, `name`, or `createdAt`. A minus sign sorts from highest or latest first (`?sort=-price`), and several fields can be combined with commas (`?sort=-price,title`). Any other field returns 400 Bad Request, so clients cannot sort by fields that do not exist.

```
const SORTABLE_FIELDS = ["price", "title", "name", "createdAt"];

for (const field of sort.split(",").map((f) => f.trim())) {
  const key = field.replace(/^-/, "");
  if (!SORTABLE_FIELDS.includes(key)) return sendError(res, HttpStatus.BAD_REQUEST, ...);
  sortBy[key] = field.startsWith("-") ? -1 : 1;
}
sortBy._id = 1; // keeps the order stable between pages
```

Requests: `GET /api/v1/products?sort=price` (Postman 5.4, lowest price first) and `?sort=-price` (5.5, highest price first).

Box 1 below: sorting code in productController.js. **Box 2 below:** Postman output of GET /api/v1/products?sort=price.

Part 5: API Testing

Using Postman/Insomnia REST, test every endpoint (*Provide screenshots per function,* Add more pages if necessary**).**

Verify that:

- Products can be created.
- Products can be retrieved.

- Products can be updated.
- Products can be deleted.
- Categories function correctly.
- Suppliers function correctly.
- Pagination returns the expected records.
- Filtering returns the correct products.
- Sorting orders the products correctly.
- Appropriate HTTP status codes are returned.

Every endpoint was tested in Postman with the collection **Activity3-CTADWEBL** (`Activity3-CTADWEBL.postman_collection.json`, 58 requests). Each request has automated tests that check the status code, the standard JSON format, and the returned data, and all of them pass (Test Results tab). The collection saves the token and IDs in collection variables, and folder 8 deletes the temporary category, supplier, and product that it created.

Requirement	Postman requests	Result
Products can be created	4.1 POST /api/v1/products	201 Created; category and supplier populated
Products can be retrieved	4.2 GET /api/v1/products/campus-trail-mix, 5.1 GET /api/v1/products	200 OK
Products can be updated	4.3 PUT price and stock; 4.4–4.6 assign suppliers	200 OK with the new values
Products can be deleted	8.3 DELETE /api/v1/products/campus-trail-mix, then 8.4 GET	200 OK, then 404 Not Found
Categories function correctly	2.1–2.6, 8.1, 8.5, 8.7	List, create, get, update, delete; 409 for a duplicate or a category still in use; 404 when missing
Suppliers function correctly	3.1–3.6, 8.2, 8.6	List, create, get, update, delete; 400 invalid email; 409 duplicate or still in use
Pagination returns the expected records	5.1, 5.2	5 products on page 1, the rest on page 3; totalPages = total ÷ limit rounded up
Filtering returns the correct products	5.3, 6.1, 6.2	Only the matching category, supplier, and price range
Sorting orders the products correctly	5.4, 5.5	Prices in ascending and descending order
Appropriate HTTP status codes are returned	All [Validation] requests	400, 401, 404, and 409 as listed below

HTTP status codes returned for invalid requests

#	Request	Status and message
1.2	POST /api/v1/users with an email that is already registered	409 Conflict – Email already registered
1.4	POST /api/v1/sessions with a wrong password	401 Unauthorized – Invalid email or password
1.6	GET /api/v1/users/me without a token	401 Unauthorized – No token provided

2.5	POST /api/v1/categories with an existing name	409 Conflict – name "Snacks" already exists
2.6	GET /api/v1/categories/000000000000000000000000	404 Not Found – Category not found
3.5	POST /api/v1/suppliers with email "not-an-email"	400 Bad Request – Please enter a valid email address
4.7	POST /api/v1/products without a title	400 Bad Request – Product title is required
4.10	POST /api/v1/products without a token	401 Unauthorized – No token provided
4.11	GET /api/v1/products/no-such-product	404 Not Found – Product not found
5.6	GET /api/v1/products?page=0	400 Bad Request – page must be a whole number of 1 or more
5.7	GET /api/v1/products?sort=color	400 Bad Request – Cannot sort by "color"
7.3	POST /api/v1/carts for a user who already has a cart	409 Conflict – This user already has a cart
8.1	DELETE /api/v1/categories/:id while products still use it	409 Conflict – Cannot delete a category that still has products

Users and sessions

[Paste screenshot here: 1.1 Sign up – 201 Created]

[Paste screenshot here: 1.3 Sign in – 200 OK with token]

[Paste screenshot here: 1.5 My profile – 200 OK]

Products

[Paste screenshot here: 4.1 Create – 201 Created]

[Paste screenshot here: 4.2 Get by name – 200 OK]

[Paste screenshot here: 4.3 Update – 200 OK]

[Paste screenshot here: 8.3 Delete – 200 OK]

[Paste screenshot here: 8.4 Get after delete – 404 Not Found]

Categories

[Paste screenshot here: 2.1 Get all – 200 OK]

[Paste screenshot here: 2.2 Create – 201 Created]

[Paste screenshot here: 2.3 Get by id – 200 OK]

[Paste screenshot here: 2.4 Update – 200 OK]

[Paste screenshot here: 8.5 Delete – 200 OK]

Suppliers

[Paste screenshot here: 3.1 Get all – 200 OK]

[Paste screenshot here: 3.2 Create – 201 Created]

[Paste screenshot here: 3.3 Get by id – 200 OK]

[Paste screenshot here: 3.4 Update – 200 OK]

[Paste screenshot here: 8.6 Delete – 200 OK]

Pagination, filtering, and sorting

[Paste screenshot here: 5.1 / 5.2 Pagination]

[Paste screenshot here: 5.3 Filtering by category]

[Paste screenshot here: 5.4 / 5.5 Sorting by price]

Carts, orders, and reviews

[Paste screenshot here: 7.1 Create cart – 201 Created]

[Paste screenshot here: 7.4 Place order – 201 Created]

[Paste screenshot here: 7.6 Write review – 201 Created]

Status codes

[Paste screenshot here: 1.2 / 2.5 / 7.3 – 409 Conflict]

[Paste screenshot here: 1.4 / 1.6 / 4.10 – 401 Unauthorized]

[Paste screenshot here: 3.5 / 4.7 / 5.6 – 400 Bad Request]

[Paste screenshot here: 2.6 / 4.11 – 404 Not Found]

[Paste screenshot here: 8.1 / 8.2 – 409 Conflict when deleting a record still in use]

Part 6: Reflection Questions

Answer the following questions:

1. **Why is RESTful architecture widely used in modern web applications?**
2. **Why should resource names be used instead of action verbs in REST endpoints?**
3. **How do HTTP status codes improve communication between the client and server?**
4. **What are the advantages of implementing pagination in an API?**
5. **How do filtering and sorting improve the usability of an application such as Bulldogs Exchange?**

1. Why is RESTful architecture widely used in modern web applications?

REST uses the standard parts of HTTP: URLs for resources, methods for actions, status codes for results, and JSON for data. Any client that can send an HTTP request (a React website, a mobile app, or Postman) can use the same API without special libraries. REST is also stateless: every request carries what the server needs, such as the JWT in the Authorization header, which makes the server easier to scale and to deploy on platforms like Vercel. Because these conventions are shared across the industry, REST APIs are predictable, easy to document and test, and their responses can be cached. In Bulldogs Exchange, the same /api/v1/products endpoint can serve the React storefront today and a mobile app later.

2. Why should resource names be used instead of action verbs in REST endpoints?

In REST, the URL identifies a thing (a resource) and the HTTP method already says what to do with it. GET /api/v1/products reads products, POST creates one, PUT /api/v1/products/:name updates it, and DELETE removes it, so one noun covers all four operations. Verbs in URLs such as /getProducts, /createProduct, or /deleteProduct repeat what the method already says, multiply the number of URLs, and make every developer name endpoints differently. Using nouns keeps the API consistent and easy to guess, which is why this activity replaced /signup and /signin with POST /api/v1/users and POST /api/v1/sessions.

3. How do HTTP status codes improve communication between the client and server?

A status code tells the client the result of a request before it even reads the body, in a way every HTTP tool understands. 2xx means success (200 OK, 201 Created), 4xx means the client has to fix something (400 invalid data, 401 not signed in, 404 not found, 409 conflict with existing data), and 5xx means the server failed. The client can react correctly without parsing error text; for example, the storefront can send the user to the sign-in page on 401 or show "name already taken" on 409. Named constants such as HttpStatus.CONFLICT also make the server code easier to read and keep the codes consistent across all controllers.

4. What are the advantages of implementing pagination in an API?

Pagination returns data in small pages instead of the whole collection at once. The database reads fewer documents, responses are smaller and faster, and the server uses less memory, which matters more as Bulldogs Exchange adds products. The client can show the first page right away and load more only when the user asks. Returning total and totalPages lets the frontend build page buttons, and the maximum limit of 100 protects the server from requests that try to download everything in one call.

5. How do filtering and sorting improve the usability of an application such as Bulldogs Exchange?

Filtering and sorting let students find what they need quickly instead of scrolling through every product. A student can show only Uniforms, only items from one supplier, or only products within a budget (minPrice and maxPrice), then sort by lowest price or search for a keyword like "jacket". The server does this work with indexed queries, so results are fast and only the needed products are sent. Because every option can be combined in one request, the same endpoint can power the category pages, the search bar, and the price filter of the storefront.

Part 7: Enhancement Activity

Do the following:

- Create Version 1 API routes. (e.g. /api/v1/product).
- Modify the Product API to support filtering using multiple query parameters. (e.g. GET /api/v1/products?category=Uniform&supplier=Nike)
- Implement keyword searching (e.g. GET /api/v1/products?search=jacket)

- Allow pagination, filtering, sorting, and keyword search to work together. (e.g. GET /api/v1/products?page=1&limit=5&category=Uniform&sort=price&search=jacket)
- Modify every Product API response to follow a consistent JSON structure. (See below for sample response)
- Provide Screenshots of your Source Code and Output, add more pages if necessart

Example response:

```
{
  "success": true,
  "message": "Products retrieved successfully.",
  "count": 15,
  "data": [...]
}
```

All routes are now served under **/api/v1**, and the Product API supports pagination, filtering with several parameters, keyword search, and sorting in a single request. Every response, for products and for every other resource, uses the same JSON structure.

Version 1 API routes

```
const API_V1 = "/api/v1";
app.use(`${API_V1}/products`, productRoutes);
app.use(`${API_V1}/categories`, categoryRoutes);
app.use(`${API_V1}/suppliers`, supplierRoutes);
app.use(`${API_V1}/users`, userRoutes);
app.use(`${API_V1}/sessions`, sessionRoutes);
app.use(`${API_V1}/carts`, cartRoutes);
app.use(`${API_V1}/orders`, orderRoutes);
app.use(`${API_V1}/reviews`, reviewRoutes);
```

Requests to an unversioned or unknown path return 404 with a JSON message. A future `/api/v2` can be added next to v1 without breaking clients that still use v1.

Multiple filters, keyword search, and all features together

Keyword search uses the text index on `title` and `content` created in Activity 2, so it matches words in the product title or description, including word variations such as jacket and jackets. In this catalog the jacket belongs to the **Apparel** category, so the combined request uses `category=Apparel` instead of "Uniform" from the example.

#	Request (GET)	Result
6.1	/api/v1/products?category=Uniforms&supplier=Bulldog Threads	NU Uniform Polo only (both filters applied)
6.2	/api/v1/products?supplier=Bulldog Threads&minPrice=700	Hoodie Jacket and Bulldogs Jersey
6.3	/api/v1/products?search=jacket	Hoodie Jacket (keyword search)
6.4	/api/v1/products?page=1&limit=5&category=Apparel&sort=price&search=jacket	Hoodie Jacket, page 1 of 1
6.5	/api/v1/products?page=1&limit=2&supplier=Bulldog Threads&sort=-price	Hoodie Jacket (1199) and Bulldogs Jersey (899); total 3, totalPages 2

6.6	/api/products (no version)	404 Not Found – the unversioned route no longer exists

Consistent JSON response

```
{
  "success": true,
  "message": "Products retrieved successfully.",
  "count": 2,
  "total": 3,
  "page": 1,
  "limit": 2,
  "totalPages": 2,
  "data": [
    { "name": "hoodie-jacket", "title": "Hoodie Jacket", "category": { "name": "Apparel" },
      "supplier": { "name": "Bulldog Threads" }, "price": 1199, "stock": "Preorder", ... },
    { "name": "bulldogs-jersey", "title": "Bulldogs Jersey", "category": { "name": "Sports" },
      "supplier": { "name": "Bulldog Threads" }, "price": 899, "stock": "Preorder", ... }
  ]
}

// Error response
{ "success": false, "message": "Product not found" }
```

Field	Meaning
success	true when the request worked, false when it failed
message	a short, readable description of the result
count	number of items in data (lists only)
total, page, limit, totalPages	pagination details (product list only)
data	the document or list of documents (left out on errors)

[Paste screenshot here: index.js – version 1 routes]

[Paste screenshot here: productController.js – getProducts (pagination, filters, search, sort)]

[Paste screenshot here: utils/apiResponse.js – standard response format]

[Paste screenshot here: 6.1 Multiple filters – category and supplier]

[Paste screenshot here: 6.2 Multiple filters – supplier and price range]

[Paste screenshot here: 6.3 Keyword search – jacket]

[Paste screenshot here: 6.4 Pagination + filtering + sorting + search together]

[Paste screenshot here: 6.5 All features together – 2 per page, highest price first]

[Paste screenshot here: 6.6 Unversioned route – 404 Not Found]

Conclusion

The Bulldogs Exchange API now follows RESTful design standards: resource-based plural URLs, HTTP methods for actions, versioned routes under /api/v1, named HTTP status code constants, and one consistent JSON response format. The Product API supports pagination, filtering by category, supplier, and price, keyword search, and sorting, and all of them work together in one request. Postman tests

confirmed that every endpoint, including full CRUD for products, categories, and suppliers, returns the expected data and status codes.

Activity Rubrics

Criteria	Beginning (1)	Developing (2)	Proficient (3)	Excellent (4)	Pts
API Versioning Implementation	API versioning was not implemented or the route structure contained major errors.	API versioning was partially implemented but some routes were incorrectly configured or inaccessible.	API versioning was correctly implemented with only minor issues in route organization or configuration.	API versioning was fully implemented following RESTful best practices, with all versioned routes functioning correctly.	
Advanced Filtering and Keyword Search	Filtering and search features were not implemented or failed to return the expected results.	Basic filtering or search was implemented, but multiple query parameters or keyword searches produced inconsistent results.	Filtering and keyword search were implemented correctly with only minor issues affecting functionality.	Advanced filtering and keyword search were fully implemented, accurately processing multiple query parameters and returning the expected results.	
Combined Query Functionality	Pagination, filtering, sorting, and searching could not be used together or produced incorrect results.	Some query features worked together, but combinations resulted in inconsistent or incomplete responses.	Most query features worked together correctly with only minor issues.	Pagination, filtering, sorting, and keyword search were fully integrated and functioned correctly in a single request.	
Standardized API Responses and Testing	API responses were inconsistent or testing was incomplete with several endpoint failures.	API responses followed a standard format in some endpoints, but testing revealed multiple issues.	Most API responses followed a consistent structure, and testing demonstrated correct functionality with minor issues.	All API responses followed a standardized JSON format, and comprehensive Postman testing confirmed that every endpoint functioned correctly.	
Documentation and Submission Requirements	Required screenshots and documentation were largely incomplete, missing, or poorly organized.	Most documentation and screenshots were submitted but contained several omissions or lacked organization.	Documentation and screenshots were complete with only minor formatting or organizational issues.	All required screenshots, source code, and documentation were complete, accurate, well-organized, and professionally presented.	
Total Score					